One-Dimensional Bin Packing Problem: An Experimental Study of Instances Difficulty and Algorithms Performance

Chapter · March 2021

DOI: 10.1007/978-3-030-68776-2_10

3 authors:

Guadalupe Carmona-Arroyo
Universidad Veracruzana
6 PUBLICATIONS **15** CITATIONS
SEE PROFILE

Jenny Betsabé Vázquez-Aguirre
Universidad Veracruzana
3 PUBLICATIONS **3** CITATIONS
SEE PROFILE

Marcela Quiroz
Universidad Veracruzana
47 PUBLICATIONS **375** CITATIONS
SEE PROFILE

One-Dimensional Bin Packing Problem: An experimental study of instances difficulty and algorithms performance

Guadalupe Carmona-Arroyo[1], Jenny Betsabé Vázquez-Aguirre[1], and Marcela Quiroz-Castellanos[1]

Universidad Veracruzana, Artificial Intelligence Research Center, Xalapa, Mexico,
gcarmonaarroyo@gmail.com, jennybey13@gmail.com, maquiroz@uv.mx

Abstract. The one-dimensional Bin Packing Problem (BPP) is one of the best-known optimization problems, and it has a significant number of applications. For this reason, several strategies have been proposed to solve it, but only few works have focused on the study of the characteristics that distinguish the BPP instances and that could affect the performance of the algorithms that solve it. In this work, we present a comprehensive study of the performance of four well-known algorithms on a set of new BPP instances. First, the features of the instances and the performance of the algorithms are quantified by indices; next, an exploratory analysis of the indices is carried out in order to identify the characteristics that define the difficulty of a BPP instance and to understand the performance of the algorithms. The algorithmic behavior explanations obtained by the study suggest that the difficulty of the BPP instances is related to: (1) the bin capacity; (2) the dispersion of the items weights; and (3) the difference between the largest and the smallest items weight.

Keywords: Bin Packing · Instance difficulty · Algorithmic performance · Characterization

1 Introduction

Throughout the search for the best possible solutions for NP-hard problems, a wide variety of solution procedures have been proposed, however, there is no efficient algorithm capable of finding the best solution for all possible instances of a problem. For the development of better solution methods, a profound comprehension of the properties of problem instances and the performance of the algorithms that solve them is needed.

This work is focused on a classical combinatorial optimization problem: the one-dimensional Bin Packing Problem (BPP). The BPP consists of packing a set of items with a certain size within a set of bins that have the same capacity, the goal is to minimize the number of bins used to store all the items. BPP has been one of the most studied combinatorial optimization problems, and a significant

number of strategies have been proposed to solve it, such as exact algorithms, approximate algorithms, heuristics, as well as metaheuristic algorithms.

The main idea of this work arises from the knowledge that, despite the existence of multiple solution techniques for this problem, few investigations have centered on the analysis of the behavior of the algorithms. In other words, good solutions have been obtained with the proposed strategies, but there are not enough studies about why some instances of the problem can be solved more easily than others. To address this issue, this chapter focuses on the characterization of the difficulty of the BPP instances and the relationship of these characteristics with the final result of the algorithms through exploratory analysis processes.

As part of this experimental analysis, three classic heuristics and a state-of-the-art grouping genetic algorithm are implemented to solve this problem on a set of 2800 new instances. This implementation aims to analyze the results obtained, also studying the characteristics of the test instances. For this, the properties of the instances and the performance of the algorithms are quantified using indices. The three classic heuristics are First Fit Decreasing (FFD), Best Fit Decreasing (BFD), and Minimum Bin Slack (MBS). The first two are simple heuristics and are often part of more complex algorithms. Minimum Bin Slack is a sophisticated heuristic that has given good results in experiments, it has inspired new heuristics and local searches. The last algorithm is the Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT), which is one of the best in the literature for BPP. This analysis has allowed us to identify some characteristics that define the difficulty of BPP instances and to understand the performance of the algorithms. Some of the more revealing features are the capacity of the bins and the ranges of the weights of the items. We could observe that the four studied algorithms decrease their effectiveness as the bin capacity increases, which confirms the relationships found in previous studies. Furthermore, we were able to confirm that instances with average weights around 0.3 of the bin capacity seem to be the hardest for simple heuristics. On the other hand, MBS and GGA-CGT were the algorithms with the best results, however, their execution times were affected by the increase in the bin capacity and by the range of the weights vector.

The document continues as follows: Section 2 presents the Bin Packing Problem, the indices, and the instances that are used for the proposed analysis. Section 3 describes the algorithms implemented to study their performance. Section 4 presents the results obtained by these algorithms according to their four performance measures. Then, in Section 5 the performance analysis of the algorithms is presented. And finally, Section 6 presents conclusions and future research directions.

2 The Bin Packing Problem

The one-dimensional Bin Packing Problem (BPP) is a well-known grouping combinatorial optimization problem defined as follows: given an unlimited number

of bins with a fixed capacity c and a set of n items, each with a specific weight w_i, BPP aims to find the minimum number of bins needed to pack all the items without violating the capacity of any bin.

Formally, given a set $N = 1, \ldots, n$ of items, a set of bins with the same capacity $c > 0$, and let w_i be the respective weight of each item $i \in N$ with $0 < w_i \leq c$. BPP consists of grouping all the elements in bins in such a way that the sum of their weights does not exceed c, and the number of bins m used is minimal [1]. In other words, we want to create a partition of N into the minimum number of subsets B_j possible:

$$\bigcup_{j=1}^{m} B_j = N \tag{1}$$

such that:

$$\sum_{i \in B_j} w_i \leq c, \quad 1 \leq j \leq m, \quad i \in N$$

BPP belongs to the NP-hard class [3, 4] because it requires a significant amount of computational resources to be solved, and the problem complexity grows exponentially with the problem size since the number of possible partitions is higher than $(n/2)^{n/2}$. This implies there is no efficient algorithm to find an optimal solution for every instance of BPP.

This problem has a significant number of applications in the industry, so it has been highly studied, and multiple algorithms have been developed to solve it, from approximate and exact algorithms to metaheuristics. The performance of the algorithms has been evaluated with different types of published problems. However, for understanding which features of the instances match the strengths and weaknesses of different algorithms, there is much more work to be done.

Table 1 shows the description of the notation used in this work.

Table 1. Notation.

Notation	**Description**
n	Number of items
c	Bin capacity
opt	Number of bins in the optimal solution
w_i	Weight of the item i $(i = 1, ..., n)$
$l(j)$	Accumulated weight in bin j
Z	List of items sorted in decreasing order according w_i
Sol	Solution, i.e., complete assignment of items to bins
m	Number of bins in the solution
$s(j)$	Free space in the bin j, i.e., $c - l(j)$
N	Set of n items
W	Set of n weights

2.1 Instances

For the BPP study, most algorithms proposed in the literature have been evaluated using a well-studied trial benchmark [6]; it includes 1615 instances in which the number of items n varies within $[50, 1000]$, the bin capacity c is within $[100, 100000]$ and the ranges of the weights are within $(0, c]$.

Some researchers have investigated the impact of the weight vector, the bin capacity c, and the number of items n in the hardness of the instances [5, 11]. Studies about the effect of the range of the weight w_i of the items have shown that a smaller range makes the instances more difficult for fast approximation heuristics, which have low performance on instances with weights distributed between 0.15 and 0.5 of the bin capacity c. Other ranges also create difficult instances when the average weight is between 0.3 and 0.5 of the bin capacity c. On the other hand, instances with average weights greater than 0.5 of the bin capacity seem to be the easiest for most of BPP state-of-the-art algorithms. Instances with average weights less than 0.3 of the bin capacity have also been shown to be easy for algorithms that include random strategies. Another important parameter to consider when studying BPP instances is the bin capacity c, the analysis of the performance of the algorithms have shown that instances characterized by very large capacities are generally difficult to solve. The studies have also pointed out that one of the features that has a greater impact on the BPP instances difficulty is associated with the optimal solutions structure, instances whose optimal solution presents very little free space in bins are very difficult.

To contribute with the study of the features that define the difficulty of a BPP instance, we have performed some experimental comparisons over new instances with different range of the weights w_i and different bin capacities c. All the instances were generated with known optima: in any optimal solution, all the bins must be filled without leaving unused space (perfect packing). We generate 2800 instances, which are referred to as BPPv_u_c, and they are available at https://github.com/gcarmonaar/CO. The weights were generated in the interval $(0, v_u c]$ so that v_u correspond to the upper bound on the weights, relative to the bin capacity c. The instances can be grouped in four classes with $v_u = 0.25$, $v_u = 0.5$, $v_u = 0.75$ and $v_u = 1$, respectively. There are seven sets in each class, each consists of 100 instances with bin capacities $c = 10^2$, $c = 10^3$, $c = 10^4$, $c = 10^5$, $c = 10^6$, $c = 10^7$, and $c = 10^8$, respectively. The first class (BPP.25 class) includes instances where the number of items n varies within $[110, 154]$ and the number of bins in the optimal solution is 15; the second class (BPP.5 class) comprises instances where the number of items n varies within $[124, 167]$ and the number of bins in the optimal solution is 30; the third class (BPP.75 class) contains instances where the number of items n varies within $[132, 165]$ and the number of bins in the optimal solution is 45; and the fourth class (BPP1 class) involves instances where the number of items n varies within $[148, 188]$ and the number of bins in the optimal solution is 60. The difficulty of the proposed instances is caused by the lack of any slack in the optimal solution and by the large capacities of the bins. We summarize this information in Table 2 in which

the number of instances per class (#), the distribution of the items weights (w_i), the bin capacities (c), the number of items (n), and the number of bins in the optimal solution (opt) are shown.

Table 2. Description of the instances.

CLASS	**#**	$\boldsymbol{w_i}$	$\boldsymbol{c}$	$\boldsymbol{n}$	$\boldsymbol{opt}$
BPP.25	700	(0, 0.25c]	$10^2, 10^3, 10^4, 10^5, 10^6, 10^7, 10^8$	[110, 154]	15
BPP.5	700	(0, 0.5c]		[124, 167]	30
BPP.75	700	(0, 0.75c]		[132, 165]	45
BPP1	700	(0, c]		[148, 188]	60

2.2 Index description

Most of the works focused on analyzing the performance of heuristic algorithms are concentrated on comparative analysis of experimental results. However, to understand and explain the behavior of an algorithm, it is necessary to identify and study the factors that affect it, as mentioned in some works that highlight the importance of explaining the reason for the observed results [8, 9, 11].

The difficulty of the BPP instances has been previously studied. Different authors have proposed characterization indices; a study of the BPP structure was presented by Cruz [10] who formulated a set of indices to measure the characteristics of BPP. On the other hand, Quiroz [11] presents in her work a set of indices to measure the difficulty of BPP, which allow to identify trends on weights as well as important information on their distribution. Cruz and Quiroz make an extensive study on the selection of the indices that provide the most relevant information of the instances and that present a greater relationship with the performance of the algorithms. We have selected some of them after preliminary tests, and the selected ones are shown in Table 3.

2.3 Performance measures

In order to measure the algorithms performance, we have evaluated three characteristics for the algorithms.

First, we have calculated the relative difference between the optimal (opt) and the number of bins (m) obtained by an algorithm. This measurement tells us how close to the optimal is a solution found by an algorithm. The above is defined by the Equation 2.

$$Dif = \frac{m - opt}{opt} \tag{2}$$

Table 3. Indices presented by Cruz [10] and Quiroz [11].

Equation	Description
$t = \frac{\sum_{i=1}^{n} \frac{w_i}{n}}{c}$	Capacity occupied by an average item.
$d = \sqrt{\frac{\sum_{i=1}^{n} [t - (\frac{w_i}{c})]^2}{n}}$	Degree of dispersion of the quotient of the weight of the items and the bin capacity.
$f = \frac{\sum_{i=1}^{n} factor(c, w_i)}{n}$	Proportion of items whose weight w_i is a factor of the bin capacity; where $factor(c, w_i)$ is 1 when $(c \mod w_i) = 0$ and is 0 otherwise.
$b = \begin{cases} 1 & \text{if } c \geq \sum_{i=1}^{n} w_i \\ \frac{c}{\sum_{i=1}^{n} w_i} & \text{otherwise} \end{cases}$	Proportion of the total weight than can be assigned in one bin with capacity c.
$Range = \max(w_i) - \min(w_i)$	Difference between the largest and the smallest weight.
$RC = \frac{Range}{c}$	Range of the weights relative to the bin capacity c.
$Major = \frac{\max(w_i)}{c}$ $Minor = \frac{\min(w_i)}{c}$	The largest and smallest weight of the instance, respectively, regarding the bin capacity.
$Multiplicity = \frac{\sum_{i=1}^{n'} m_i}{n'}$	The average number of repetitions for each weight; where m_i is the number of items each with different weight.
$Major\_Multiplicity = \max(M)$ $Minor\_Multiplicity = \min(M)$	Represent the maximum and minimum frequency of weight recurrence for the n items, respectively. Where M saves the frequency for each different weight.
$Pct\_\tilde{n} = \frac{\tilde{n}}{n}$	Proportion of large items; where $\tilde{n}$ is the number of items with weight larger than fifty percent of the bin capacity, i.e., $w_i > c/2$.
$L2$	Lower bound for BPP solutions [1, 2].

Another measurement is made through the cost function introduced by Falkenauer and Delchambre [14]. This function evaluates the average of the squaring of the bin efficiency given by $l(j)/c$, which measures the exploitation of bins capacity. Equation 3 shows this calculation.

$$F_{BPP} = \frac{\sum_{j=1}^{m}(l(j)/c)^2}{m} \tag{3}$$

In addition, we have measured the execution time of the algorithms ($time$). This measurement is given in seconds and the experiments were performed on a computer with an Intel (R) Core (TM) i5 CPU with 2.50 GHz. and Microsoft Windows 10.

It is important to note that F_{BPP} and Dif are measures of effectiveness since they give us information about the solutions quality and $time$ is a measure of efficiency, by providing us with information on the speed of the algorithms.

3 Algorithms

For the solution of BPP, elaborate procedures that incorporate various techniques have been designed. Algorithms proposed in the literature range from simple heuristics to hybrid strategies, including branch and bound techniques [7], evolutionary algorithms [12] and special neighborhood searches [19].

In the present work, four well-known methods to solve BPP have been analyzed with the new instances. Three of the methods are simple heuristics that have been incorporated into high performance algorithms and the other is one of the best state-of-the-art algorithms. These algorithms are mentioned below.

3.1 First Fit Decreasing (FFD)

First Fit Decreasing (FFD) is a classical heuristic to solve BPP based on the FF strategy [15], it is a simple heuristic that has been used in more complex strategies, such as creating initial solutions in evolutionary algorithms. This algorithm works as follows: before starting to place the items, they are sorted in non-increasing order of their sizes. Then, this heuristic packs each item in the first bin with enough capacity. If no bin is found, FFD creates a new bin to pack the item. This algorithm can be implemented to have a running time of $O(n \log n)$. Algorithm 1 shows FFD process.

Algorithm 1: First Fit Decreasing.

```
Input: Z,c
Output: m
for each item i ∈ Z do
    for each bin j ∈ Sol do
        if w_i + l(j) ≤ c then
            Pack item i in bin j
            Update l(j)
            Break for
    if item i was not packed in any available bin then
        Create a new bin for item i
```

3.2 Best Fit Decreasing (BFD)

Best Fit Decreasing (BFD) is another of the most popular heuristics to solve BPP [15] and like FFD, it has been used in several more complex algorithms as an aid to skew solutions. It consists of assigning each item to the bin that has the maximum load, where the item fits. If several bins have the maximum load, then the first one found is chosen. In the same way as the previous heuristic, it considers the items in decreasing order of the weights. BFD can be implemented in $O(n \log n)$ time. The above procedure is described in the Algorithm 2.

Algorithm 2: Best Fit Decreasing.

```
Input: Z,c
Output: m
for each item i ∈ l do
    best_bin = ∅
    for each bin j ∈ Z do
        if w_i + l(j) ≤ c and l(j) > l(best_bin) then
            best_bin = j
    if no bin has enough capacity to pack i then
        Create a new bin to pack i
    else
        Pack item i in the fullest bin (best_bin)
        Update l(j)
```

3.3 Minimum Bin Slack (MBS)

Minimum Bin Slack (MBS) was proposed by Gupta and Ho [16]. It is a bin-focused heuristic; at each execution of MBS, an attempt is made to find a set of items that fits the bin capacity as much as possible [18]. Fleszar and Hini [17] have presented a recursive implementation of MBS. At each step, the items that

have not been packed are considered in non-increasing order of their weights, then, this procedure tests all possible subsets of items and the subset whit the least free space is chosen. If the algorithm finds a subset of items whose sum of weights equals the bin capacity, then the search stops. Algorithm 3 shows the procedure to fill a bin, it begins whit $A^* = A = \emptyset$ and $q = 1$, where A^* represents the best subset (according to $l(A)$), A is the auxiliary subset to evaluate. The procedure is repeated removing the chosen items for a bin until the list Z is empty. The complexity of this procedure is $O(2^n)$.

Algorithm 3: Minimum Bin Slack.

```
   Input: q = 1
   Output: A*
 1 for r = q to n do
 2     Let i be the r−th element ∈ Z
 3     if w_i ≤ s(A) then
 4         A = A ∪ {i}
 5         Apply MBS(r + 1)
 6         A = A \ {i}
 7         if s(A*) = 0 then
 8             End
 9 if s(A) ≤ s(A*) then
10     A* = A
```

3.4 GGA-CGT

The Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT) is a method proposed by Quiroz-Castellanos et al. [12] to solve BPP. It is one of the best algorithms found in the state-of-the-art to solve this problem; it focuses on the transmission of the best genes on the chromosomes, keeping a balance between selective pressure and diversity in the population, to favor the generation and evolution of high-quality solutions. This algorithm was executed once with the parameters that the author have established. See the procedure in Algorithm 4.

Algorithm 4: GGA-CGT.

```
 1 Generate an initial population P with FF−ñ
 2 while generation < max_gen and Size(best_solution > L_2) do
 3     Select n individuals to cross by means of Controlled_Selection
 4     Apply Gene_Level_Crossover + FFD to the n_c selected individuals
 5     Apply Controlled_Replacement to introduce progeny
 6     Select n_m individuals and clone elite solutions by means of
         Controlled_Selection
 7     Apply Adaptive_Mutation + RP to the best n_m individuals
 8     Apply Controlled_Replacement to introduce clones
 9     Update the global_best_solution
```

4 Results

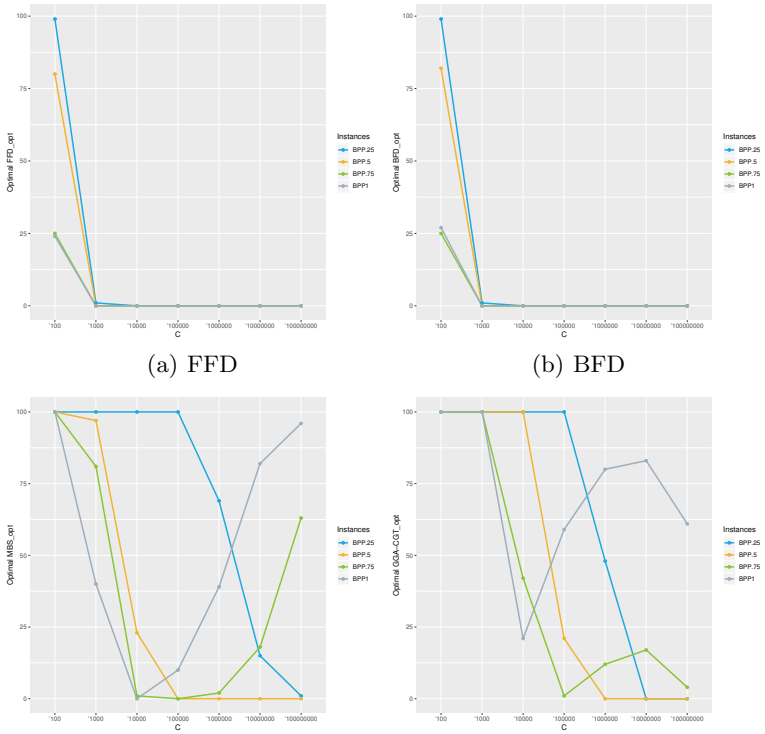
As we mentioned before, in this study we want to identify the relationships between the characteristics of the BPP instances and the behavior of the algorithms. For this, in Section 2.3, we have described the performance measures of the algorithms, which are F_{BPP}, Dif, and $time$. Therefore in this section we present the results of the executions of the four algorithms described in the previous section. In addition, the number of optimal solutions obtained by each algorithm is presented here.

Table 4. Number of optimal solutions found by each algorithm.

CLASS	SET	FFD	BFD	MBS	GGA-CGT
BPP.25	100	99	99	100	100
	1000	1	1	100	100
	10000	0	0	100	100
	100000	0	0	100	100
	1000000	0	0	69	48
	10000000	0	0	15	0
	100000000	0	0	1	0
TOTAL BPP.25		**100**	**100**	**485**	**448**
BPP.5	100	80	82	100	100
	1000	0	0	97	100
	10000	0	0	23	100
	100000	0	0	0	21
	1000000	0	0	0	0
	10000000	0	0	0	0
	100000000	0	0	0	0
TOTAL BPP.5		**80**	**82**	**220**	**321**
BPP.75	100	25	25	100	100
	1000	0	0	81	100
	10000	0	0	1	42
	100000	0	0	0	1
	1000000	0	0	2	12
	10000000	0	0	18	17
	100000000	0	0	63	4
TOTAL BPP.75		**25**	**25**	**265**	**276**
BPP1	100	24	27	100	100
	1000	0	0	40	100
	10000	0	0	0	21
	100000	0	0	10	59
	1000000	0	0	39	80
	10000000	0	0	82	83
	100000000	0	0	96	61
TOTAL BPP1		**24**	**27**	**367**	**504**
TOTAL		**229**	**234**	**1337**	**1549**

In Table 4, we have the number of instances that each algorithm solved optimally. It contains the total number of instances resolved by class and also a general total, that is, how many of the 2800 instances were solved. As we can see, FFD solves a total of 229 instances of the 2800, and BFD solves 234. On the other hand, MBS solves a total of 1337 and finally GGA-CGT solves 1549.

From the table, it can also be observed that between FFD and BFD there is a difference of 5 instances in which the optimal solution was found. In the first class of BPP.25 instances, there are a total of 100 instances (99 from the first set), and for the other classes, the optimal solutions number decreases for both algorithms, with the slight difference that BFD solves five more. It is important to mention that all the instances that FFD solves, BFD does too. For MBS and GGA-CGT a similar behavior is seen for the sets in which the solution was found. It should be noted that in the BPP.25 class, MBS solves more instances, but in the remaining three classes GGA-CGT solves more.



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 1. Number of optimal solutions found by each algorithm.

Furthermore, Figure 1 shows the four graphs of the optimal ones found by each algorithm. On the horizontal axis are the sets, and the vertical axis has the

number of instances that were optimally solved. The colors correspond to the 4 main classes of instances. We see that FFD (Figure 1(a)) solves instances only for the capacity c =100 in all four classes, in the same way as BFD (Figure 1(b)). However, MBS (Figure 1(c)) solves multiple instances where the bin capacity is lower, as well as several with the larger values of c. GGA-CGT (Figure 1(d)) has similar behavior than MBS, with the difference that in 3 of the 4 main classes this algorithm does not solve several instances with the largest bin capacities.

Table 5. Relative Difference sum between each algorithm and the best solution.

CLASS	SET	FFD	BFD	MBS	GGA-CGT
BPP.25	100	0.067	0.067	0	0
	1000	6.6	6.6	0	0
	10000	6.667	6.667	0	0
	100000	6.667	6.667	0	0
	1000000	6.667	6.667	2.067	3.467
	10000000	6.667	6.667	5.667	6.667
	100000000	6.667	6.667	6.6	6.667
BPP.5	100	0.667	0.6	0	0
	1000	3.333	3.333	0.1	0
	10000	3.333	3.333	2.567	0
	100000	3.333	3.333	3.333	2.633
	1000000	3.333	3.333	3.333	3.333
	10000000	3.333	3.333	3.367	3.333
	100000000	3.333	3.333	3.333	3.333
BPP.75	100	1.667	1.667	0	0
	1000	2.222	2.222	0.422	0
	10000	2.222	2.222	2.244	1.289
	100000	2.222	2.222	2.311	2.2
	1000000	2.222	2.222	2.267	1.956
	10000000	2.222	2.222	1.933	1.844
	100000000	2.222	2.222	0.933	2.133
BPP1	100	1.267	1.217	0	0
	1000	1.667	1.667	1	0
	10000	1.667	1.667	1.683	1.317
	100000	1.667	1.667	1.567	0.683
	1000000	1.667	1.667	1.017	0.333
	10000000	1.667	1.667	0.317	0.283
	100000000	1.667	1.667	0.067	0.65
TOTAL		**85.28**	**85.17**	**46.13**	**42.12**

As we have mentioned before, the relative difference between the number of bins of the solutions found by the four algorithms and the optimal was calculated through Equation 2. Table 5 shows the total relative difference Dif per set, each row has the sum of the relative differences for the 100 instances of the set. Note than a Dif value equal to zero, since we are presenting the sum, means that the

100 instances of the set were optimally solved. Thus, at the end of the columns for each algorithm, we present the sum of the relative differences of the 2800 instances.

An important aspect to mention is that the relative difference measures how greater is the number of bins found by an algorithm compared to the optimal solution. So when the value of Dif grows, we can affirm that the solutions of the algorithms are further away from the optimal solution. We can see that the largest value of the total sum of the relative difference is presented by FFD, although the difference with BFD is very small. Furthermore, the smallest value is given by GGA-CGT, where most zeros are also found.

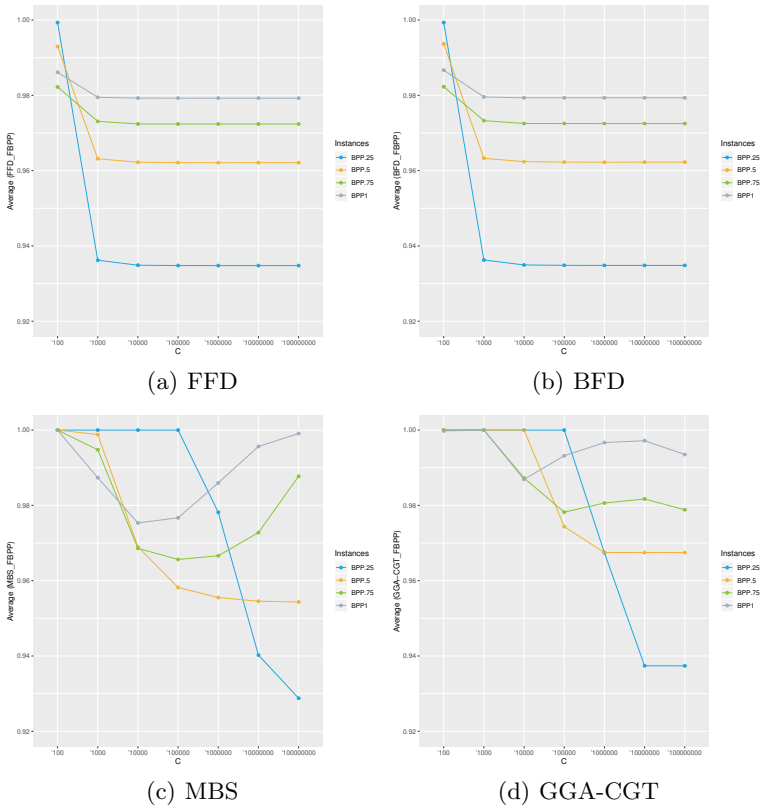
Table 6. F_{BPP} average for the four algorithms.

CLASS	SET	FFD	BFD	MBS	GGA-CGT
BPP.25	100	0.999	0.999	1	1
	1000	0.936	0.936	1	1
	10000	0.935	0.935	1	1
	100000	0.935	0.935	1	1
	1000000	0.935	0.935	0.977	0.967
	10000000	0.935	0.935	0.94	0.937
	100000000	0.935	0.935	0.929	0.937
BPP.5	100	0.993	0.993	1	1
	1000	0.963	0.963	0.998	1
	10000	0.962	0.962	0.969	1
	100000	0.962	0.962	0.958	0.994
	1000000	0.962	0.962	0.956	0.967
	10000000	0.962	0.962	0.954	0.967
	100000000	0.962	0.962	0.954	0.967
BPP.75	100	0.982	0.982	1	1
	1000	0.973	0.973	0.995	1
	10000	0.972	0.973	0.968	0.987
	100000	0.972	0.973	0.966	0.991
	1000000	0.972	0.972	0.967	0.981
	10000000	0.972	0.972	0.973	0.982
	100000000	0.972	0.972	0.987	0.979
BPP1	100	0.986	0.987	1	1
	1000	0.979	0.98	0.987	1
	10000	0.979	0.979	0.975	0.987
	100000	0.979	0.979	0.977	0.995
	1000000	0.979	0.979	0.986	0.997
	10000000	0.979	0.979	0.996	0.997
	100000000	0.979	0.979	0.999	0.994
AVERAGE		**0.966**	**0.966**	**0.979**	**0.987**

On the other hand, we have Table 6 with the average F_{BPP} (Equation 3) for the solutions found. Note that if F_{BPP} equals 1, it means that the optimum was

found for the 100 instances, in addition to the fact that all these instances have an optimal solution with a 100% of fullness rate. And as this number decreases, we can argue that the average fill of bins does as well. This can be seen in the results for MBS and GGA-CGT, where the value of 1 is found in the rows where the algorithm solved all instances. That is not the case for FFD and BFD, where for none of the sets all the optimal ones were found. At the end of the table the average of the total instances is shown.

Figure 2 presents the average value of F_{BPP} for each type of different bin capacities (sets). The different colors represent the four classes of instances. The horizontal axis contains the bin capacities, and the vertical axis shows the average value of the F_{BPP} function. Each graph corresponds to a different algorithm.



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 2. Average of the F_{BPP} value for the solutions of the four algorithms.

As before, we observe similar behavior in FFD and BFD (Figures 2(a) and 2(b)), which is a constant value from capacity 10^4 in the four classes of instances. The value of F_{BPP} for MBS solutions decreases in three of the classes and then

grows again. Contrary to GGA-CGT, where the performance decrease in classes BPP.25 and BPP.5 without recovering, and both have a variable behavior. Note that the BPP.25 class (with the blue color) has the most similar behavior in the last two algorithms.

Table 7 presents the execution times in seconds of each algorithm (average). At the end of the table, the average time spent by each algorithm is shown. Note that in the case of MBS and GGA-CGT, there are high execution times compared to simple heuristics. Also, this behavior occurs only in sets with larger bin capacities. GGA-CGT shows the highest average CPU time. As we mentioned earlier, *time* is a measure of efficiency, and although the MBS and GGA-CGT algorithms have presented the best values in terms of effectiveness, the execution time is too high compared to the simple FFD and BFD heuristics. Note that the highest value occurs in the set of $c = 10^8$ in the BPP.25 class with the MBS heuristic.

Table 7. Average CPU time in seconds for the four algorithms.

CLASS	SET	FFD	BFD	MBS	GGA-CGT
BPP.25	100	0.003	0.003	0.037	0.002
	1000	0.003	0.003	0.037	0.026
	10000	0.003	0.003	0.047	0.436
	100000	0.003	0.003	0.148	10.92
	1000000	0.003	0.003	1.631	36.96
	10000000	0.003	0.004	12.52	27.48
	100000000	0.003	0.004	75.97	22.71
BPP.5	100	0.008	0.007	0.044	0.014
	1000	0.008	0.007	0.062	0.218
	10000	0.007	0.007	0.123	3.718
	100000	0.007	0.007	0.696	17.32
	1000000	0.007	0.007	4.631	15.28
	10000000	0.007	0.007	34.25	13.78
	100000000	0.007	0.007	67.32	12.34
BPP.75	100	0.010	0.023	0.042	0.023
	1000	0.010	0.023	0.071	0.309
	10000	0.010	0.023	0.101	6.163
	100000	0.010	0.023	0.359	9.232
	1000000	0.010	0.023	1.37	10.39
	10000000	0.010	0.023	4.702	10.02
	100000000	0.010	0.024	13.74	10.07
BPP1	100	0.012	0.030	0.043	0.044
	1000	0.012	0.030	0.049	0.407
	10000	0.013	0.030	0.108	6.137
	100000	0.013	0.030	0.282	5.191
	1000000	0.013	0.030	0.757	4.756
	10000000	0.013	0.030	0.664	4.245
	100000000	0.013	0.030	0.689	10.32
AVERAGE		**0.007**	**0.016**	**7.875**	**8.518**

With the tables and figures above, we were able to observe certain trends in the algorithms and their performance evaluation in terms of effectiveness and efficiency. Now, in the next section, these results will be studied to determine if they could be an effect of the characteristics of the BPP instances.

5 Experimental Analysis

In this section, using an exploratory approach, we seek insights into the relationships between the BPP instances structure and the performance of the BPP algorithms. The relationships are studied by applying correlation and graphical analysis to have an explanation of the results obtained by the above algorithms. We have used the indices presented in Section 2 to analyze the existing correlation between the BPP features, the relative difference, the cost function, and the execution time for each algorithm. Figure 3 presents the correlation matrix, which includes the degree of linear relationship between the features of the instances and the algorithms performance. In the rows, we have the BPP indices (note that we have also included the number of items n, as well as the bin capacity c). In the columns, we have the relative difference Dif for the four algorithms, then the cost function F_{BPP} and finally the $time$. It is important to remember that a good value for the relative difference Dif is equal to zero and it grows as the error of the algorithm increases. However, the F_{BPP} value has a different behavior considering as optimal, the values equal to 1.

From Figure 3 we can see that the highest correlations are associated with the relative difference Dif and the cost function F_{BPP} obtained by FFD and BFD. Remember that these are the algorithms that obtained the lowest results in the number of optimal solutions found. From the results presented in Table 4, it can be observed that as the average weights of the items and the bin capacity increase, the probability of these algorithms to find the optimal solutions decrease. We can also appreciate that the time for FFD presents a null correlation against some characteristics of the instances.

The values of the correlations in Figure 3 suggest that, regardless of the algorithm, the relationships between the characteristics of the instances with Dif and F_{BPP}, always have the same meaning. These relationships are stronger in the simple FFD and BFD heuristics, which seems to be the consequence of the fact that these heuristics do not include strategies focused on maximizing the filling of the bins, as opposed to MBS and GGA-CGT.

The correlation matrix also shows which characteristics are more related to the values of F_{BPP} and Dif. For example, the correlations associated with $L2$, $Major$, RC, t, and d, show that as these are higher, the values of the relative difference decrease. This behavior is seen in the result of the four algorithms, but it is only strong for the result of FFD and BFD. In the same way, when the values of these metrics increase, the F_{BPP} value does so, maintaining a consistent value of association for FFD and BFD. To have a clearer notion of this description, we have plotted some of these indices to show the behavior mentioned. First, we

decided to plot 3D graphs with the bin capacity c, the $Major$ index, and the value of F_{BPP}.

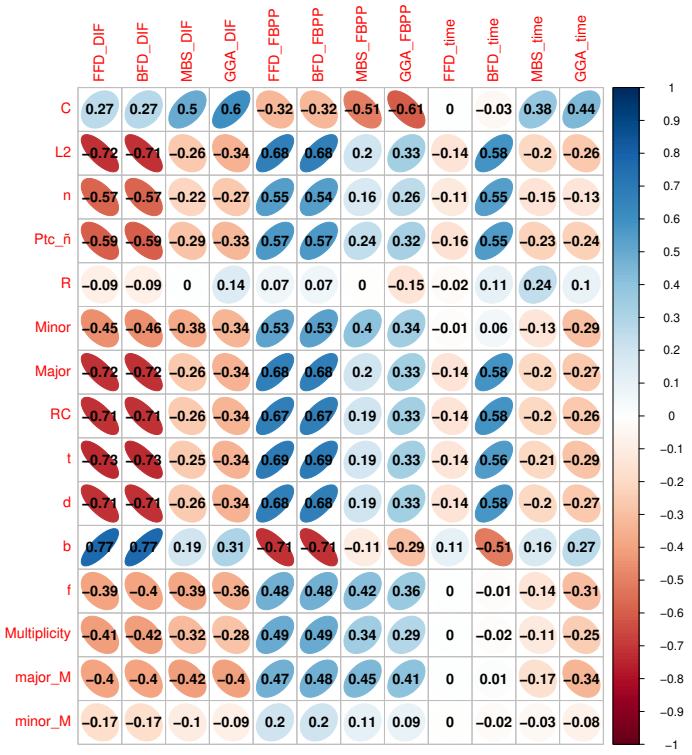
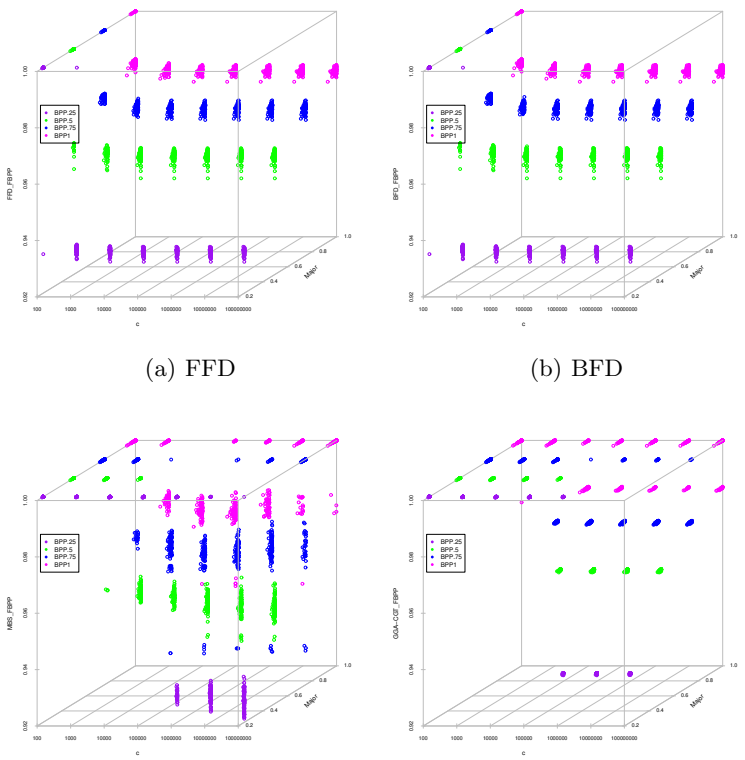


Fig. 3. Correlation matrix between the BPP indices and the three performance measures for the 2800 instances.

Figure 4 contains four 3D scatter plots, one for each algorithm; in each plot, the x−axis indicates the bin capacity c, the y−axis depicts the value of the index $Major$, and the z−axis indicates the cost function F_{BPP} of the solutions generated for each instance. A different color is used to plot each instance according to the class it belongs, as can be seen in this figure, the instances of each set are together, according to the bin capacity c. From the figure, we can see that as the value of $Major$ increases, the value of F_{BPP} also increases; that is, as the value of the highest weight in the instance relative to the bin capacity is higher, FFD and BFD have a higher value of F_{BPP} (Figures 4(a) and 4(b)). Furthermore, these algorithms are the ones with the least dispersion in the values obtained per set. Also, we can note that MBS (Figure 4(c)) presents the greatest variability in F_{BPP} values, both by class and by set.

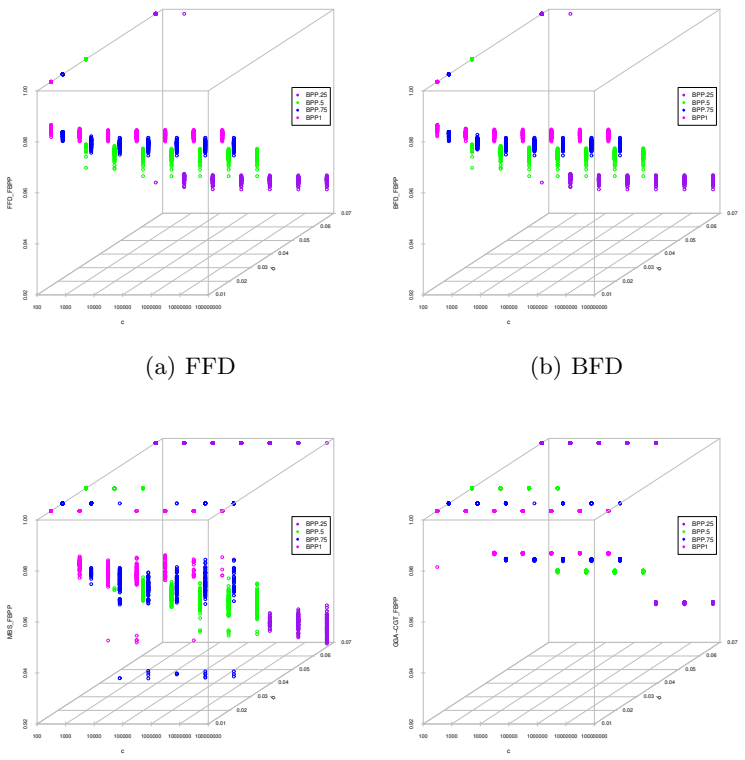


(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 4. Relations between the index $Major$, the bin capacity c, and F_{BPP} for the 2800 instances.

Similarly, Figure 5 shows the impact of the index b in the values of the cost function F_{BPP} obtained for each instance. This figure has the same structure of Figure 4. Here, the y−axis indicates the b values of each instance. As seen in this figure, when the proportion of the total weight than can be assigned to a bin b grows, the F_{BPP} value decreases, especially for FFD and BFD (Figures 5(a) and 5(b)), which are the algorithms with the least number of optimal solutions found. In the case of MBS (Figure 5(c)), the dispersion associated with the value of the metric is very wide, both in the classes and in the sets. Finally, GGA-CGT (Figure 5(d)) shows a similar behavior regarding the relation between b and F_{BPP}, however, for this algorithm most of the results are closer to the optimal solution, which indicates that characteristic b is not a problem for the algorithm to find the optimal solution.



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 5. Relations between the index b, the bin capacity c and, F_{BPP} for the 2800 instances.

The previous results allowed us to graphically appreciate that the result of the algorithms is different by class and set, therefore, we have done the correlation analysis for each class of instances. The following subsections present the particular analysis of the instances of each of the classes. In each of the cases, we present the correlation analysis between the indices and the performance measures, with the difference that these correlations are only measured for the 700 instances of the respective class. It is important to clarify that some correlations could not be extracted because the value of some of the characteristics is the same for all the instances in the same class. Question marks (?) in the correlation matrix denote that situation. Also, we have chosen the f index (given the correlations presented) to have a graphical analysis of its behavior in each of the classes.

5.1 Class BPP.25

The class BPP.25 includes 7 sets of instances, each with 100 instances for every bin capacity c $(10^2, \ldots, 10^8)$, where the number of items n varies within $[110, 154]$, the weights are distributed between $(0, 0.25c]$ and the number of bins in the optimal solution is 15. For this class, the BPP indices that are proportional to the bin capacity ($Minor$, $Major$, RC, t and d) present similar values, the average values are: $Minor = 0.001$, $Major = 0.246$, $RC = 0.246$, $t = 0.115$ and $d = 0.072$. This class was the least difficult for the three heuristics (FFD, BFD and MBS), the algorithms found a higher number of optimal solutions than in the other classes.

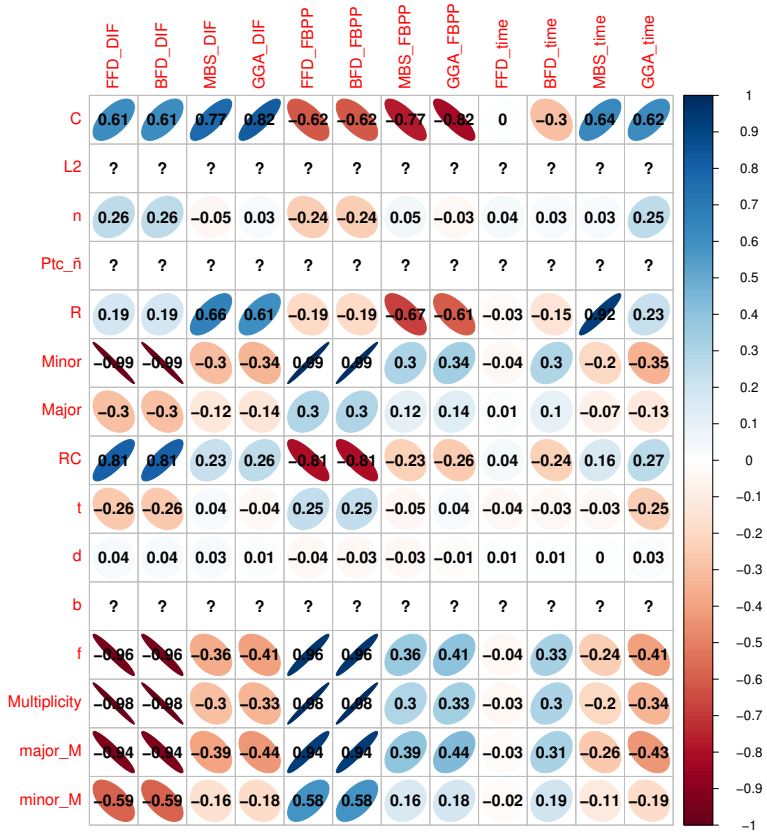
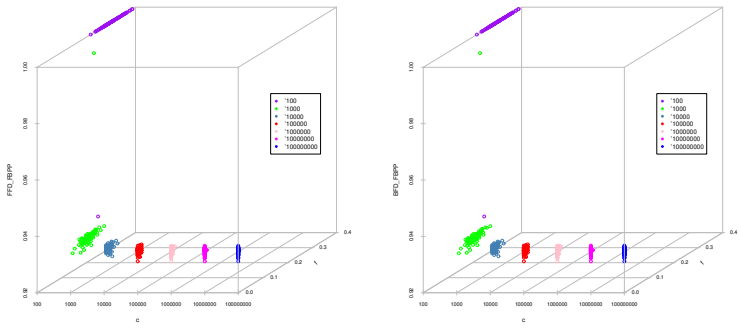


Fig. 6. Correlation matrix between the BPP indices and the three performance measures for the class BPP.25.

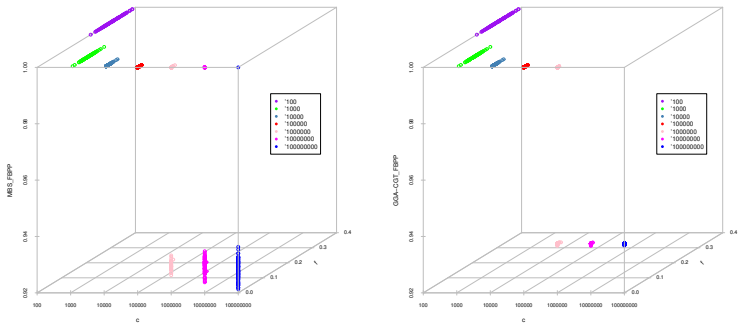
The correlation analysis for this class shows interesting results (Figure 6), for example, we can see that as $Minor$, f, $Multiplicity$, $Major\_Multiplicity$, and

Minor_Multiplicity increase, the FFD, BFD and MBS algorithms obtain better results. However, when the bin capacity c increases, these algorithms perform less well. In the same way, the algorithms MBS and GGA-CGT are affected by the bin capacity. An increasing time for MBS is also notable when the difference between the largest and the smallest weight R grows. This could be attributed to the fact that it is more difficult for the algorithm to find the appropriate grouping of items for each bin.

To have a visual analysis of these relations, Figure 7 present the scatters plots for the proportion of items whose weight is a factor of the bin capacity f, the bin capacity c and the cost function F_{BPP} achieved by each algorithm. In this figure, a different color is used for each of the seven sets.



(a) FFD (b) BFD



(c) MBS (d) GGA-CGT

Fig. 7. Relations between the index f, the bin capacity c, and F_{BPP} for the instances in the BPP.25 class.

From Figure 7 we can observe how for the smallest bin capacity $c = 10^2$ the four algorithms found most of the optimal solutions, without being affected by

the value of f. However, as the bin capacities increase, the effectiveness of FFD and BFD (Figures 7(a) and 7(b)) drops dramatically; unlike MBS (Figure 7(c)) which even solved some instances of all sets. On the other hand, GGA-CGT (Figure 7(d)), despite being the algorithm that solves more instances on the other classes, did not manage to find an optimal solution in none of the two sets with the highest bin capacities ($c = 10^7$ and $c = 10^8$).

5.2 Class BPP.5

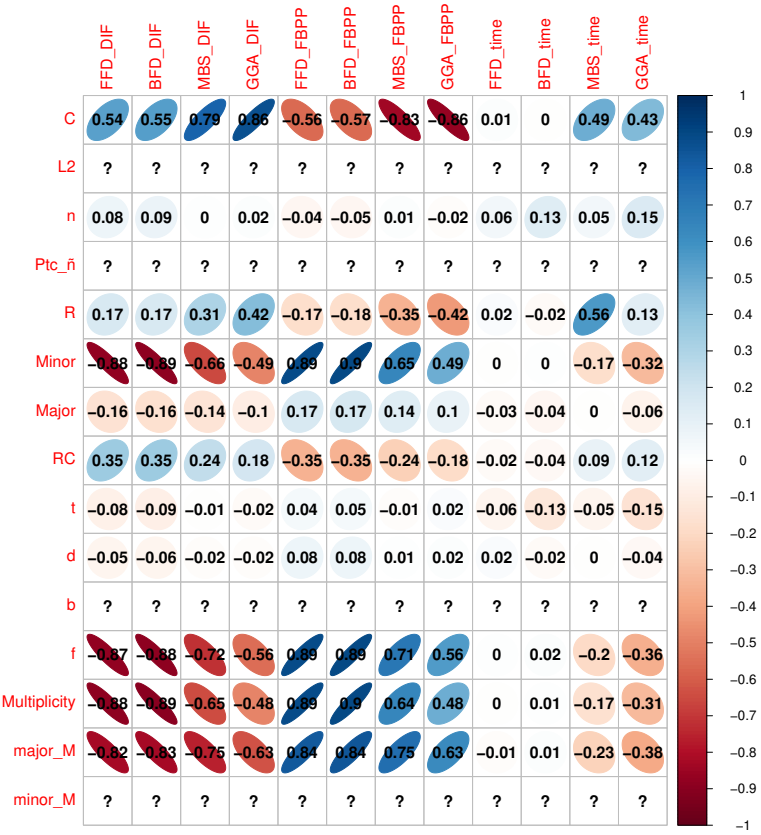
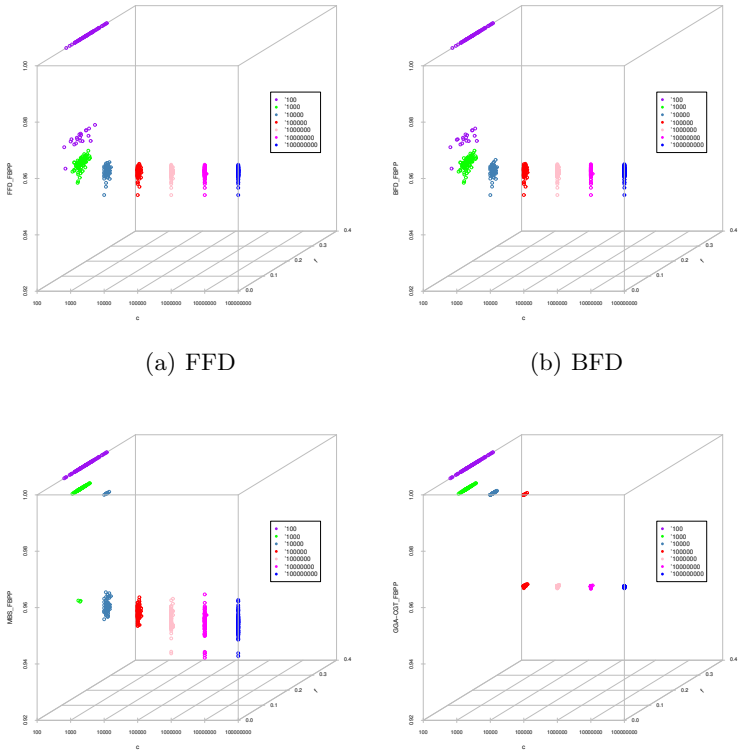


Fig. 8. Correlation matrix between the BPP indices and the three performance measures for the class BPP.5.

For the class BPP.5, the number of items n varies within $[124, 167]$, the weights are distributed between $(0, 0.5c]$ and the number of bins in the optimal solution is 30. For this class, the BPP indices that are proportional to the bin capacity ($Minor$, $Major$, RC, t and d) present similar values, the average values are:

$Minor = 0.001$, $Major = 0.495$, $RC = 0.493$, $t = 0.213$ and $d = 0.140$. This class was the hardest for MBS heuristic, the algorithm only found 220 optimal solutions, the lowest number in the four classes. For the instances of this class, the correlation matrix (Figure 8) shows that the relationships between the BPP indices and the performance of the algorithms are like those observed for the class BPP.25. However, it is evident that, for some indices, the associative level begins to be lower, although it does not decrease dramatically.



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 9. Relations between the index f, the bin capacity c, and F_{BPP} for the instances in the BPP.5 class.

Similar to Figure 7, Figure 9 present the scatters plots for f, c and F_{BPP}, for the class BPP.5 and we can see that for the characteristic f and the F_{BPP} value, there is a greater variability, especially for the FFD and BFD algorithms (Figures 9(a) and 9(b)). On the other hand, it can be seen that the F_{BPP} values are more consistent by class and set since only for FFD and BFD there are two groups of F_{BPP} values obtained, one optimal and the other very low, this being

similar for all sets. For its part, GGA-CGT (Figure 9(d)) presents almost zero variability within the sets, which indicates that the algorithm allows a better exploitation of the bin's capacity, which is what F_{BPP} represents.

5.3 Class BPP.75

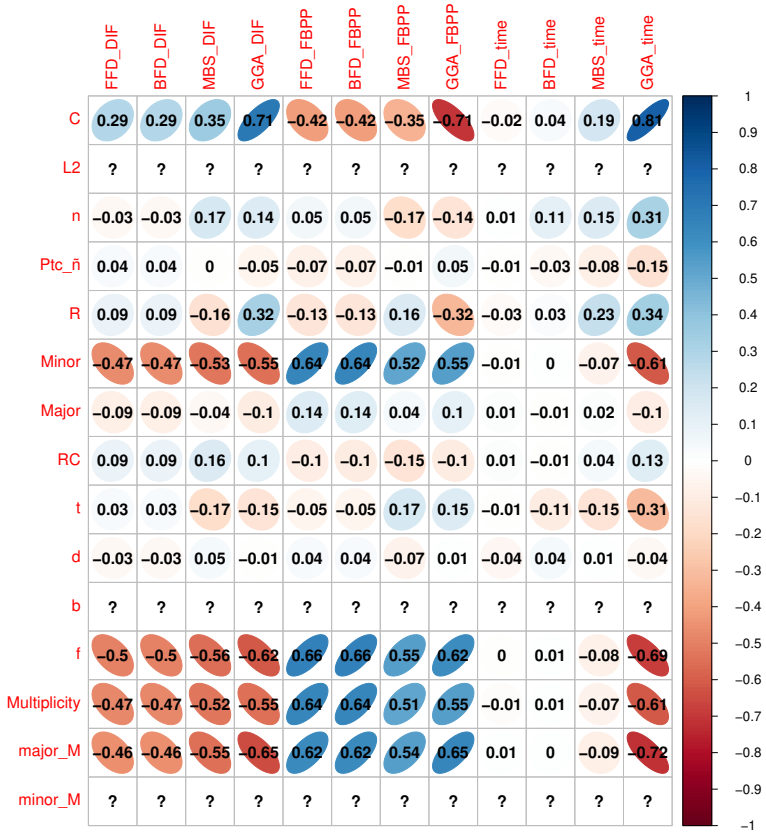
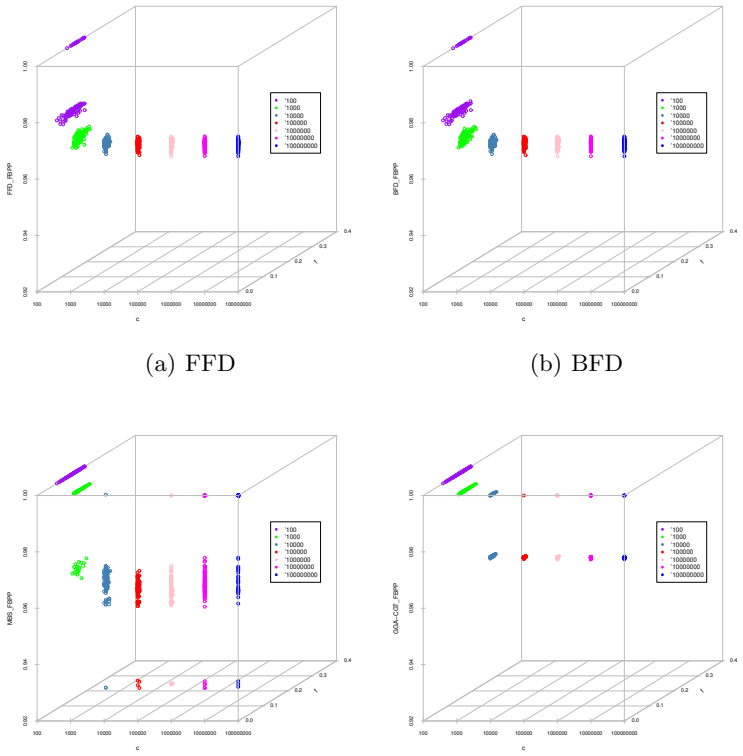


Fig. 10. Correlation matrix between the BPP indices and the three performance measures for the class BPP.75.

For the class BPP.75, the number of items n varies within $[132, 165]$, the weights are distributed between $(0, 0.75c]$ and the number of bins in the optimal solution is 45. For this class, the BPP indices that are proportional to the bin capacity ($Minor$, $Major$, RC, t and d) present similar values, the average values are: $Minor = 0.001$, $Major = 0.741$, $RC = 0.739$, $t = 0.301$ and $d = 0.202$. This class was the hardest for GGA-CGT, the algorithm only found 276 optimal

solutions, the lowest number in the four classes. Figure 10 presents the correlation matrix for this class, the efficiency of the algorithms, measured by the index *time*, seems to be influenced by the features of the BPP instances. The value of *time* decreases when $Minor$, f, $Multiplicity$, $Major\_Multiplicity$, and $Minor\_Multiplicity$ increase for the GGA-CGT algorithm. However, it increases when the bin capacity increases. It is remarkable how the correlations for the rest of the algorithms and characteristics have decreased against the first two classes. It is important to observe than even in this set of instances the F_{BPP} value is compromised when the bin capacity increases, especially for the GGA-CGT algorithm.



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 11. Relations between the index f, the bin capacity c, and F_{BPP} for the instances in the BPP.75 class.

The graphs for class BPP.75 in the Figure 11 present a different behavior from the two previous classes. In these, it is possible to appreciate how as f increases, the value of F_{BPP} increases, finding a much greater variability in the

values obtained by this measure. It is evident that the highest values for f are found in small bin capacities, for this class the values of f are within $[0, 0.193]$, and this range does not seem to have a significant influence in the behavior of the algorithms, since the performance of these has been mainly affected by the capacity of the bins.

5.4 Class BPP1

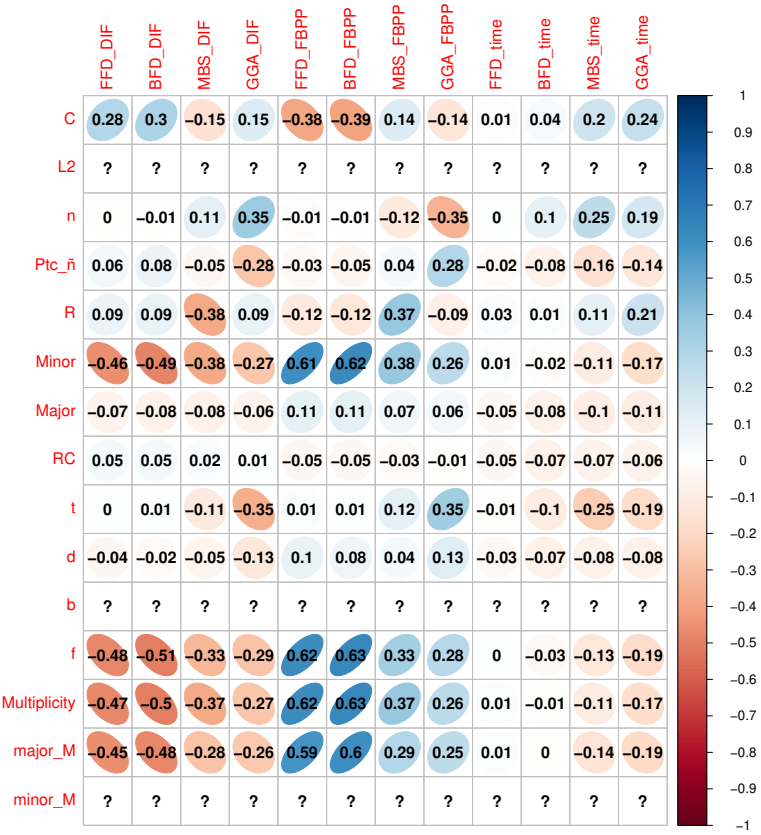
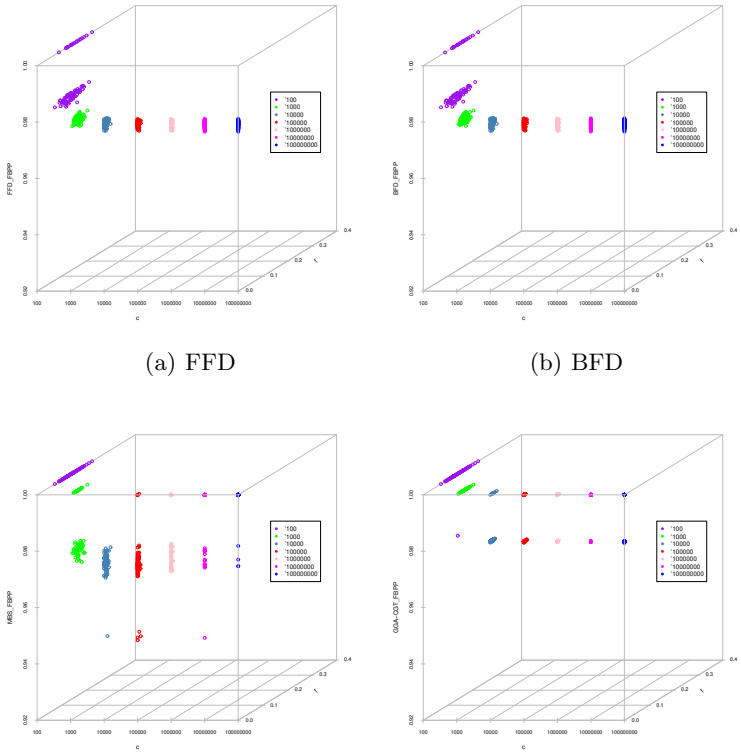


Fig. 12. Correlation matrix between the BPP indices and the three performance measures for the class BPP1.

For the class BPP1, the number of items n varies within $[148, 188]$, the weights are distributed between $(0, c]$ and the number of bins in the optimal solution is 60. For this class, the BPP indices that are proportional to the bin capacity ($Minor$, $Major$, RC, t and d) present similar values, the average values are: $Minor = 0.001$, $Major = 0.989$, $RC = 0.987$, $t = 0.368$ and $d = 0.269$. This class was

the least difficult for GGA-CGT, the algorithm found 504 optimal solutions, the highest number in the four classes. Figure 12 shows the correlation matrix for the class BPP1, as can be seen, this class presents the weakest association values between the BPP features and the performance of the algorithms, but the same characteristics are highlighted, which are: $Minor$, f, $Multiplicity$, $Major\_Multiplicity$, and $Minor\_Multiplicity$ only for FFD and BFD heuristics.

On the other hand, we have the graphs for the class in Figure 13, in this we can see that, as well as in the BPP.75 class, the bin capacity is an important characteristic that impact the effectiveness of the algorithms, since it can be observed that as the value of c increases the F_{BPP} values decrease, however, in this set we can highlight that even when the optimal solutions are not found, the F_{BPP} value is better than the other three classes, especially for MBS and GGA-CGT (Figures 13(c) and 13(d)).



(a) FFD (b) BFD

(c) MBS (d) GGA-CGT

Fig. 13. Relations between the index f, the bin capacity c, and F_{BPP} for the instances in the BPP1 class.

In the studies separated by classes, we could observe that the behavior is very different in each of them. In other words, while in one class some indices present a high correlation, in another class this may differ. We can highlight that the index *time* is the only measure that behaves similarly in all groups, without having a great correlation with the indices. As well as in each of the subsections, we were able to identify some characteristics that show high correlations with each of the classes of instances.

6 Conclusions and future work

This work addressed the characterization of new hard instances for the one-dimensional Bin Packing Problem (BPP) and the analysis of the algorithm behavior of four well-known algorithms using data analysis techniques. This study aimed to gain a deeper understanding of the difficulty of the BPP instances through the investigation of the relationships between the features of the instances and the algorithms performance. The results that were obtained by the four algorithms confirm that the new instances have a high degree of difficulty; for most of these instances, the included strategies in sophisticated algorithms like MBS and GGA-CGT do not appear to lead to better solutions. The study revealed interesting relationships between the characteristics of the new instances and the four algorithms implemented here. One of the first observations is the fact that the performance of the algorithms is affected by the bin capacity and this happens regardless of the algorithm. We have also observed that simple heuristics such as FFD and MBS did not perform well and they present the highest correlations with the characteristics of the BPP instances, in comparison with MBS and the GGA-CGT.

As we could observe, FFD and BFD presented a similar behavior in all classes of instances, both heuristics solved the least number of instances, which represents in general, the difficulty of the new instances for simple heuristics. The bin capacity and the range in which the weights are distributed were the characteristics that make these algorithms difficult. Furthermore, f, $Multiplicity$, $Major\_Multiplicity$, $Minor\_Multiplicity$ and $Minor$ indices were the most correlated with the value of the cost function and the relative difference in the results obtained by said heuristics. Another interesting result is the similar behavior presented by the MBS and GGA-CGT algorithms, which presented correlations with the bin capacity in some of the sets and classes. Despite solving a greater number of instances compared to simple heuristics, the bin capacity also affected the performance of the algorithms. We could also see that the average execution times for these algorithms is the highest, which is associated with some factors such as the range R and the bin capacity c.

Something important to mention is that these evaluated instances were created with very particular characteristics, however, the same relationships found in previous studies are observed. For example, the identification of the general difficulty of the instances when the bin capacity is large, as well as when the average weights of the items is around 0.3 of the bin capacity.

The knowledge obtained in this work and the introduction of the new sets of instances open up an interesting range of possibilities for future research. First, it is expected that the new instances presented in this work can be used for studying the performance of the state-of-the-art BPP algorithms. On the other hand, new strategies can be developed incorporating knowledge of the problem domain. Future directions also include predictions of performance and additional analysis of hard BPP instances, combining different characterization techniques to obtain better explanations regarding the internal behavior and the performance of the algorithms and the difficulty of the instances solved.

References

1. Martello S., Toth P. Knapsack Problems: Algorithms and Computer Implementations. Wiley & Sons Ltd., 1990.
2. Martello S., Toth P. Lower Bounds and Reduction Procedures for the Bin Packing Problem. Discrete Applied Mathematics, 22: 59- 70, North-Holland, Elsevier Science Publishers B.V., 1990.
3. Garey, M. R., & Johnson, D. S. (1979). Computers and intractability (Vol. 174). San Francisco: freeman.
4. Basse S. Computer Algorithms, Introduction to Design and Analysis. Editorial Addison-Wesley Publishing Company, 1998.
5. Schwerin P, Wascher G. The bin-packing problem: a problem generator and some numerical experiments with FFD packing and MTP. International Transactions in Operational Research 1997;4(5–6):377–89.
6. Delorme, M., Iori, M., & Martello, S. (2018). BPPLIB: a library for bin packing and cutting stock problems. Optimization Letters, 12(2), 235-250.
7. Delorme, M., Iori, M., & Martello, S. (2016). Bin packing and cutting stock problems: Mathematical models and exact algorithms. European Journal of Operational Research, 255(1), 1-20.
8. Chiarandini M., Paquete L., Preuss M., Ridge E. Experiments on metaheuristics: Methodological overview and open issues. Technical Report DMF-2007-03-003, The Danish Mathematical Society, 2007.
9. Snodgrass R. Ergalics: A Natural Science of Computation. February 2010.
10. Cruz, L. (2004). Caracterización de Algoritmos Heurísticos Aplicados al Diseño de Bases de Datos Distribuidas. PhD Tesis, Centro Nacional de Investigación y Desarrollo Tecnológico, Cuernavaca, Morelos, México.
11. Quiroz, M. (2014). Caracterización del proceso de optimización de algoritmos heurísticos aplicados al problema de empacado de objetos en contenedores. Instituto Tecnológico de Tijuana.
12. Quiroz-Castellanos, M., Cruz-Reyes, L., Torres-Jimenez, J., Gómez, C., Huacuja, H. J. F., & Alvim, A. C. (2015). A grouping genetic algorithm with controlled gene transmission for the bin packing problem. Computers & Operations Research, 55, 52-64.
13. Falkenauer E. The grouping genetic algorithm—widening the scope of the GAs. Belgian Journal of Operations Research, Statistics and Computer Science, 33: 79-102, 1992.
14. Falkenauer, E., & Delchambre, A. (1992, May). A genetic algorithm for bin packing and line balancing. In ICRA (pp. 1186-1192).

15. Johnson, D. S. (1974). Fast algorithms for bin packing. Journal of Computer and System Sciences, 8(3), 272-314.
16. Gupta, J. N., & Ho, J. C. (1999). A new heuristic algorithm for the one-dimensional bin-packing problem. Production planning & control, 10(6), 598-603.
17. Fleszar, K., & Hindi, K. S. (2002). New heuristics for one-dimensional bin-packing. Computers & operations research, 29(7), 821-839.
18. Fleszar, K., & Charalambous, C. (2011). Average-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem. European Journal of Operational Research, 210(2), 176-184.
19. Buljubašić, M., & Vasquez, M. (2016). Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. Computers & Operations Research, 76, 12-21.